

CE 791 HPC - Assignment 5

MPI - 1D GW Flow Equation

Andrew Navratil

The Jacobi code in Fortran was modified to implement the various versions of MPI calls for solving the inhomogeneous groundwater flow problem. The jacobi.bsub file was modified for the number of processors and used to queue execution on henry2. The Mflops were written to the fjacobi.out file and manually copied into the Matlab code after each run based on number of processors. Calculation of Mflops was updated to account for the 8 flops per call using the Jacobi approximation to the inhomogeneous problem. The Matlab code then produced the plots for Mflops vs. number of processors for each version of the code. The different MPI calls are included in the same source file, with a variable for version used to switch between them. The values for k and h were also saved to txt files on a few of the runs and plotted using Matlab to confirm the correct values were being calculated. Confirmation was performed by comparing to the analytical plots of K-field and Head field from lecture notes.

II(ii): The error value was recalculated for the inhomogeneous problem using the analytical formula. The function $f(x)$ was created in Matlab and utilized to calculate $f(0)$ and $f(10)$ in order to plug into the h formula to determine the constants c_0 and c_1 . The corresponding h formula with constants was then coded into the Fortran program for error calculation, with the average error calculated using the original square root of the sum of the squares procedure. The resulting error was printed to the screen to help verify the accuracy of the code changes. Error values for each version were all about 6.88×10^{-5} .

II(iii): The average leakage flux through the entire aquifer is about -0.05468 . This was computed on one of the runs using a second order central difference formula for $\frac{dh}{dx}$, and summing the Q values across all the points of the grid. We note that both h and k have an x dependence, which makes the analytical solution to compare a little tricky, so that work was abandoned. A first order backwards difference for $\frac{dh}{dx}$ was also tested, giving the same answer for four significant digits.

$$Q_{total} = \int_0^{10} kA \frac{dh}{dx} dx = \sum_{i=1}^n Ak(i) \frac{h(i+1) - h(i-1)}{2} dx$$

III(i): The communications in version 1 are buffer dependent and could result in a deadlock or no parallel processing. The sends and receives can be reordered so they are paired to ensure that any process will finish doing a receive before doing its own send. This can be done using the algorithm from lecture and Gropp et. al. where sends and receives are paired for even and odd processors, see version 1 (safe).

Plots for Mflops versus number of processors are shown below for each version. Each version after 0 saw an increase in performance with an increase in the number of processors. The unsafe version 1 seemed to perform the best, followed by version 3, then version 5, then version 4, then version 2. Version 0 decreased in performance with increasing number of processors after 1.

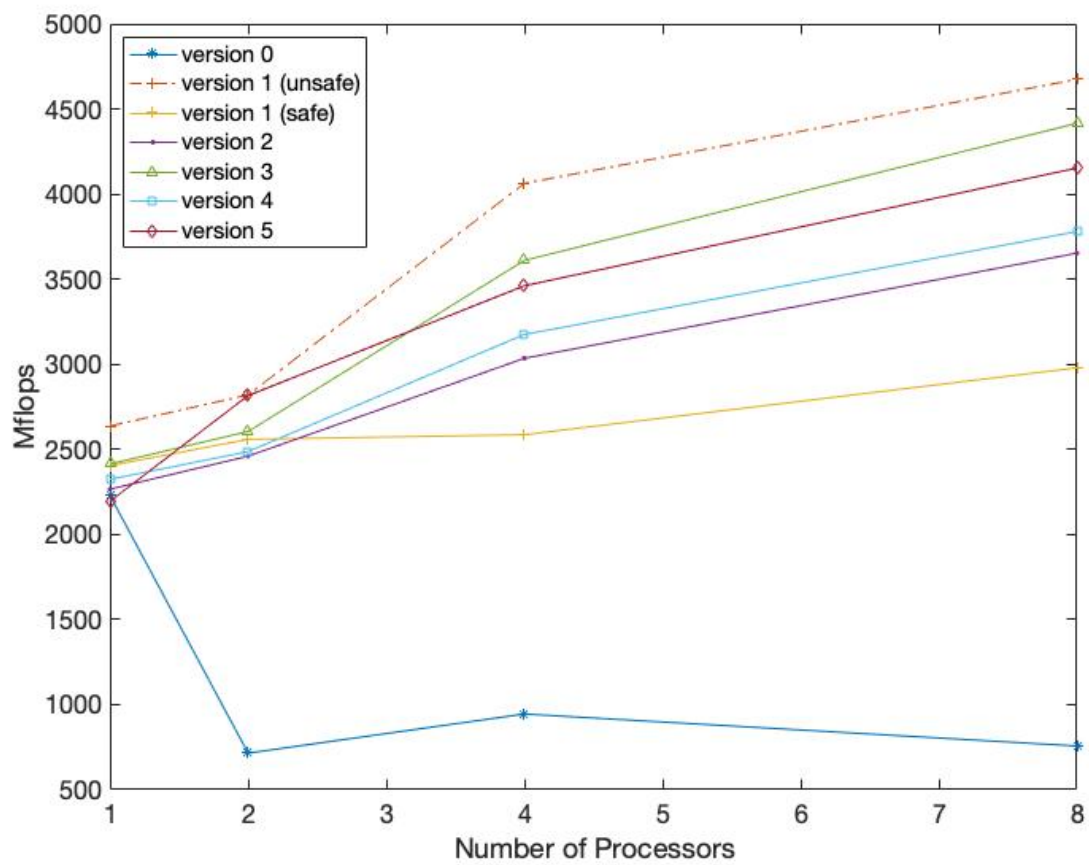


Fig. 1 version comparison on Mflops

Fortran Code: Jacobi

```
1  !=====
2  ! jacobi for 1-D Laplacian
3  !=====
4  program jacobi
5  !=====
6  include "mpif.h"
7  !=====
8
9  !=====
10 ! initialize main parameters
11 !=====
12 Integer, Parameter :: size=1000, nit = 1000000
13 !real(8), Parameter :: h0 = 1.d0, hL = 0.1d0 ! version 0
14 real(8), Parameter :: h0 = 1.0, hL = 0.0 ! version 1 BCs
15 real(8), Parameter :: x0 = 0.0, xL = 10.0 ! version 1 x domain (0-10m)
16 real(8), Parameter :: dx = (xL-x0)/size ! compute grid spacing / delta x
17 real(8), parameter :: a = 0.007, b = -0.07, c = 0.2 ! k consts
18 real(8), parameter :: c0 = -0.0055, c1 = 0.5 ! analytical sln consts
19 real(8), parameter :: qarea = 100 ! avg cross sectional area
20 integer, parameter :: ndims = 1 ! version 4 topology funcs
21 integer, parameter :: version = 5 ! version switch: 0,1,2,3,4,5
22
23 !=====
24 ! declare variables
25 !=====
26 real(8), allocatable :: h(:), hnew(:), k(:)
27 real(8) time1, time2, mflops, errs, error, sumsqerr
28 integer nprocs,i,iam,n,left,right,j
29 integer count,tag1,tag2
30 integer status(MPI_STATUS_SIZE)
31 integer commId
32 integer status_array(MPI_STATUS_SIZE, 4), req(4)
33 real(8) :: xval, fanal, Q, dhdx
34 integer :: dims(ndims)
35 logical :: isoperiodic(ndims), reorder
36 character(len=50) :: filename
37
38 !=====
39 ! initialize MPI
40 !=====
41 call MPI_INIT(ierr)
42 call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
43 call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
44
45 !=====
46 ! assume size is perfectly divisible by nprocs
47 !=====
48 n = size/nprocs
49
50 !=====
51 ! allocate arrays
```

```

52  !=====
53  allocate (h(0:n+1),hnew(0:n+1),k(0:n+1))
54
55  ! version 4 topology operation
56  if (version.eq.4) then
57      isoperiodic(1) = .false.
58      reorder = .true.
59      dims(0) = nprocs
60
61      ! this isn't working
62      !call MPI_CART_CREATE(MPI_COMM_WORLD, ndims, dims, isoperiodic, &
63          !reorder, commld, ierr)
64      ! using this version from Gropp book
65      call MPI_CART_CREATE(MPI_COMM_WORLD, 1, nprocs, .false., &
66          .true., commld, ierr)
67      call MPI_COMM_RANK(commld, iam, ierr)
68      call MPI_CART_SHIFT(commld, 0, 1, left, right, ierr)
69  end if
70
71  !=====
72  ! start timer
73  !=====
74      time1=MPI_Wtime()
75
76  !=====
77  ! initialize message passing parameters
78  !=====
79      count=1
80      tag1=1
81      tag2=2
82      !left=iam-1
83      !right=iam+1
84      if (iam.eq.0) then
85          left = MPI_PROC_null
86      else
87          left = iam-1
88      end if
89      if (iam.eq.nprocs-1) then
90          right = MPI_PROC_NULL
91      else
92          right = iam+1
93      end if
94
95  !=====
96  !initialize h and k values
97  !=====
98      if (iam.eq.0) then
99          h(0) = h0
100      end if
101      if (iam.eq.nprocs-1) then
102          h(n+1) = hL
103      end if
104
105      xval = (iam*n) * dx

```

```

106     k(0) = a*xval**2 + b*xval + c
107     xval = ((iam+1)*n + 1) * dx
108     k(n+1) = a*xval**2 + b*xval + c
109
110     do i=1,n
111         h(i) = 0.d0
112
113         xval = (iam*n + i) * dx
114         k(i) = a*xval**2 + b*xval + c
115     enddo
116
117     !=====
118     !start iterations
119     !=====
120     do j=1,nit
121         !=====
122         ! version 1
123         !=====
124         if (version.eq.1) then
125             ! version 1 unsafe
126             !! send my first grid point
127             !call MPI_Send(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
128                 !MPI_COMM_WORLD, ierr)
129             !! MPI_Ssend check for deadlock
130             !!call MPI_Ssend(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
131                 !!MPI_COMM_WORLD, ierr)
132             !! receive into right ghost buffer
133             !call MPI_Recv(h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
134                 !MPI_COMM_WORLD, status, ierr)
135             !! send my last grid point
136             !call MPI_Send(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
137                 !MPI_COMM_WORLD, ierr)
138             !! MPI_Ssend check for deadlock
139             !!call MPI_Ssend(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
140                 !!MPI_COMM_WORLD, ierr)
141             !! receive into left ghost buffer
142             !call MPI_Recv(h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
143                 !MPI_COMM_WORLD, status, ierr)
144
145             ! version 1 safe
146             ! paired send/recv by processor to avoid deadlocks
147             if (mod(iam,2).eq.0.0) then
148                 call MPI_Send(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
149                     MPI_COMM_WORLD, ierr)
150                 call MPI_Recv(h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
151                     MPI_COMM_WORLD, status, ierr)
152                 call MPI_Send(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
153                     MPI_COMM_WORLD, ierr)
154                 call MPI_Recv(h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
155                     MPI_COMM_WORLD, status, ierr)
156             else
157                 call MPI_Recv(h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
158                     MPI_COMM_WORLD, status, ierr)
159                 call MPI_Send(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &

```

```

160         MPI_COMM_WORLD, ierr)
161     call MPI_Recv(h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
162         MPI_COMM_WORLD, status, ierr)
163     call MPI_Send(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
164         MPI_COMM_WORLD, ierr)
165 end if
166
167     !=====
168     ! version 2
169     !=====
170 else if (version.eq.2) then
171     ! send to right neighbor and receive from left neighbor
172     call MPI_Sendrecv(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
173         h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
174         MPI_COMM_WORLD, status, ierr)
175     ! send to left neighbor and receive from right neighbor
176     call MPI_Sendrecv(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
177         h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
178         MPI_COMM_WORLD, status, ierr)
179
180     !=====
181     ! version 3
182     !=====
183 else if (version.eq.3) then
184     ! if not proc 0, send first grid point to left neighbor
185     if (iam.ne.0) then
186         call MPI_Send(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
187             MPI_COMM_WORLD, ierr)
188     end if
189     ! if not last proc, send last grid point to right neighbor
190     if (iam.ne.nprocs-1) then
191         call MPI_Send(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
192             MPI_COMM_WORLD, ierr)
193     end if
194
195     ! compute interior
196     do i=2,n-1
197         hnew(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
198             (k(i+1) + 2*k(i) + k(i-1))
199     enddo
200
201     ! if not proc 0, receive from left neighbor into left ghost cell
202     if (iam.ne.0) then
203         call MPI_Recv(h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
204             MPI_COMM_WORLD, status, ierr)
205     end if
206     ! if not last proc, receive from right neighbor into right ghost cell
207     if (iam.ne.nprocs-1) then
208         call MPI_Recv(h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
209             MPI_COMM_WORLD, status, ierr)
210     end if
211
212     ! compute end points
213     i = 1

```

```

214      hnew(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
215      (k(i+1) + 2*k(i) + k(i-1))
216      i = n
217      hnew(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
218      (k(i+1) + 2*k(i) + k(i-1))
219
220      !update old values
221      do i=1,n
222          h(i)=hnew(i)
223      enddo
224
225      !reinitialize boundary conditions
226      if (iam.eq.0) h(0)=h0
227      if (iam.eq.nprocs-1) h(n+1)=hL
228
229      !=====
230      ! version 4
231      !=====
232      else if (version.eq.4) then
233
234          ! send to left neighbor and receive from right neighbor
235          call MPI_Sendrecv(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
236              h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
237              commld, status, ierr)
238          ! send to right neighbor and receive from left neighbor
239          call MPI_Sendrecv(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
240              h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
241              commld, status, ierr)
242
243          !=====
244          ! version 5
245          !=====
246          else if (version.eq.5) then
247
248              call MPI_Irecv(h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
249                  MPI_COMM_WORLD, req(1), ierr)
250              call MPI_Irecv(h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
251                  MPI_COMM_WORLD, req(2), ierr)
252              call MPI_Isend(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
253                  MPI_COMM_WORLD, req(3), ierr)
254              call MPI_Isend(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
255                  MPI_COMM_WORLD, req(4), ierr)
256
257              call MPI_WAITALL(4, req, status_array, ierr)
258
259          !=====
260          end if
261          !=====
262
263          ! version 0,1,2,4,5 computation of interior (all but ver 3)
264          if (version.ne.3) then
265              !calculate new values of h
266              do i=1,n
267                  !hnew(i)=(h(i-1)+h(i+1))/2.d0 ! version 0

```

```

268         hnew(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
269         (k(i+1) + 2*k(i) + k(i-1))
270     enddo
271     !update old values
272     do i=1,n
273         h(i)=hnew(i)
274     enddo
275     !reinitialize boundary conditions
276     if (iam.eq.0) h(0)=h0
277     if (iam.eq.nprocs-1) h(n+1)=hL
278 end if
279
280 ! average leakage flux calculation (only do this one time)
281 ! approximate dh/dx using 2nd order central diff
282 !Q = 0
283 !do i = 1,n
284     !!dhdxdx = 0.5*(h(i+1) - h(i-1))
285     !dhdxdx = (h(i) - h(i-1))
286     !Q = Q + k(i) * qarea * dhdxdx * dx
287 !enddo
288 enddo
289 !=====
290 !end iterations
291 !=====
292
293 !=====
294 !end timer
295 !=====
296
297 time2=MPI_Wtime()
298 total_time=time2-time1
299
300 !=====
301 ! calculate error using analytical solution
302 !=====
303
304 errsq=0.d0
305 do i=1,n
306     !hanal=h0+(hL-h0)*(iam*n+i)/(size+1) ! version 0
307
308     xval = (iam*n + i) * dx
309     fanal = (2/sqrt(4*a*c-b*b))*atan((2*a*xval+b)/sqrt(4*a*c-b*b))
310     hanal = c0*fanal + c1
311
312     errsq = errsq + (h(i)-hanal)**2
313     !print *, "errsq = ",errsq
314 enddo
315 call MPI_REDUCE(errsq,sumsqerr,1,MPI_DOUBLE_PRECISION,MPI_SUM,0, &
316 MPI_COMM_WORLD,ierr)
317
318 !=====
319 ! print results
320 !=====
321
322 ! write out solution to file for processing in matlab
323 write(filename,"(A8,I1,A4)" 'slndata_',iam,'.txt'

```



```

322     open(1,file = filename, status='replace')
323     do i = 1,n
324         write(1,"(I5,2E20.8)") i,k(i),h(i)
325     end do
326     close(1)
327
328     if (iam == 0) then
329         !print *, "sumsqerr = ",sumsqerr
330         error=sqrt(sumsqerr)/dble(size) ! can only do this on proc 0
331
332         !mflops = 2*dble(nit)*dble(size)*1e-6/(total_time) ! version 0
333         mflops = 8*dble(nit)*dble(size)*1e-6/(total_time) ! versions 1-5
334
335         print *, "nprocs =",nprocs
336         print *, "error =",error
337         print *, 'mflops =',mflops
338         print *, 'Total time =',total_time
339
340         !print *, 'Q =',Q
341     endif
342     call MPI_FINALIZE(ierr)
343 end

```

Matlab Code

```
1 close all; clear all; clc;
2
3 if 1
4 % fjacobi bsub executed on henry2 where mflops printed to fjacobi.out and copied here
5 %   run separately for each num procs (same .out file is used)
6 procs = [1 2 4 8]
7 mflops0 = [2229 715 945 757] % version 0
8 mflops1a= [2637 2820 4063 4674] % version 1 unsafe
9 mflops1b = [2404 2559 2586 2979] % version 1 safe
10 mflops2 = [2266 2460 3035 3653] % version 2
11 mflops3 = [2414 2605 3610 4416] % version 3
12 mflops4 = [2324 2487 3175 3780] % version 4
13 mflops5 = [2195 2816 3461 4153] % version 5
14
15 figure;
16 plot(procs,mflops0,'*-','DisplayName','version 0');
17 hold on;
18 plot(procs,mflops1a,'+-','DisplayName','version 1 (unsafe)');
19 plot(procs,mflops1b,'+-','DisplayName','version 1 (safe)');
20 plot(procs,mflops2,'.-','DisplayName','version 2');
21 plot(procs,mflops3,'^','DisplayName','version 3');
22 plot(procs,mflops4,'s-','DisplayName','version 4');
23 plot(procs,mflops5,'d-','DisplayName','version 5');
24 xlabel('Number of Processors');
25 ylabel('Mflops');
26 set(gca,'FontSize',16);
27 legend('location','northwest');
28
29 end
30 %=====
31 if 0
32 % plot the k-field and head field to confirm values
33 % change v3_n1 to correct version and num procs
34 nprocs = 1
35 k=[],h=[]
36 for i = 0:nprocs-1
37     d = readmatrix(['a5/slndata_',int2str(i),'_v3_n1.txt']);
38     k = [k d(:,2)'];
39     h = [h d(:,3)'];
40 end
41
42 x0=0,xL=10,dx=(xL-x0)/1000
43 x = [dx:dx:xL];
44
45 figure;
46 plot(x,k,'r-');
47 xlabel('x (m)');
48 ylabel('k (m/d)');
49 set(gca,'FontSize',16);
50
51 figure;
```

```

52 plot(x,h,'b-');
53 xlabel('x (m)');
54 ylabel('h (m)');
55 set(gca,'FontSize',16);
56
57 end
58 %=====
59 if 0
60 % calculate c0 and c1 coefficients for analytical solution
61 % h(0)=1, h(10)=0
62 %  $4ac = 0.0056 > b^2 = 0.0049$ 
63 a = 0.007, b = -0.07, c = 0.2
64
65 f = @(x) (2/sqrt(4*a*c - b*b)) * atan( (2*a*x + b) / (sqrt(4*a*c - b*b)) )
66
67 f(0)    % = -91.4243
68 f(10)   % = 91.4243
69
70 % plugging back into  $h = c_0*f(x) + c_1$  we get
71 %  $c_0 = -0.0055$ ,  $c_1 = 0.5$ 
72
73 end

```